

Summary of Key Findings

Our research surfaced **three major technical decision points**: (1) adopting an incremental extraction pipeline akin to DBpedia-Live for real-time wiki updates, (2) employing a hybrid content+graph clustering method (e.g. Top2Vec+Node2Vec) tuned to sparse, heterophilic corpora, and (3) choosing mature AI/Visual tools for UX at scale (e.g. deck.gl/regl for 2D plots, embedding-atlas for projections). These decisions are *load-bearing* because they define our system’s continuous ingestion, analysis, and user-facing layers. For each area we drew on precedent: DBpedia-Live for updates ¹, hybrid clustering benchmarks ², and high-performance visualization documentation ³ ⁴.

Unanswered Areas: We found *no concrete reference* for a “concept-drift” UI specific to wiki topics (Q14) and *no out-of-the-box tool* visualizing sheaf-cohomology contradictions (Q15). This means we must prototype custom charts (e.g. term-frequency timelines, Sankey topic flows ⁵) and design our own “disagreement” dashboard (perhaps adapting TDA graphing). In the citation-verification space (Q16), SIDE ⁶ provides a research prototype, but no turnkey solution; we may need to pilot a mix of API checks (Crossref, Scite.ai) and human review.

Connections to Part 1: Some Part 1 results reinforce these answers. For example, our prior endorsement of **graph-based topic alignment** (Sheaf Laplacian, Fiedler) supports the hybrid clustering here, suggesting embedding+graph flows are sensible. Conversely, Part 1’s emphasis on dynamic updates aligns with Q8’s incremental model. No contradictions arose; rather, the two halves complement each other (e.g. the “learnable restriction maps” idea from Part 1 implies using knowledge graphs and reconciliation pipelines for entity alignment in Q10).

Q8: Incremental Entity/Claim Extraction

Narrative: On a page edit, we only re-run extractors for the affected page. This follows DBpedia-Live’s model: they process just the changed article and compute differences ⁷. Static data (e.g. schema mappings) remain cached (“keep” state) ⁸. We adopt a similar strategy: on each edit, mark that page’s extractors as *active* and others as *keep*. Exact-match linking (named-entity disambiguation by identifier) will typically persist unless the reference string changes; fuzzy or semantic linking (embedding-based) must be recomputed on changed text, since context may shift. We found no prior breakdown of “exact vs fuzzy vs semantic” invalidation, so we assume any changed token triggers full re-linking for that sentence. Minimal re-processing cost is therefore the cost of re-embedding the changed text chunks plus running our NER/claim pipeline on them.

Published precedent: **DBpedia-Live (Lehmann et al. 2012)** describes exactly this incremental approach. It uses a priority queue of updates, re-extracts just the altered page, and computes deltas ⁷ ¹. Its 2012 implementation processed ~108 pages/min on average ¹ (about 0.55 s/page). Our corpus is smaller (40K vs millions of enwiki), so a similar pipeline should handle dozens of edits/sec. In practice, a modern setup (GPU embedding + Python NLP) could likely process a few pages/sec; we expect **<1 second per page**. This is

peer-reviewed evidence (Lehmann) plus a **DBpedia Association blog** ⁷, so evidence is strong (one peer-reviewed study and one org. report).

Named Libraries: For extraction we can reuse or adapt DBpedia’s framework (see the DBpedia extraction toolkit on GitHub) and wikibase APIs. For NER/linking, standard libraries like **spaCy** or **Hugging Face Transformers** (for contextual NER) can be used. For claim extraction (“Claims Engine”), we likely use our in-house pipeline or open NLP tools. We’ll cache embeddings per-chunk (as we already have) so on edit only updated chunks get embedded anew with **Qwen3-Embedding**. Those vector operations use **pgvector** or similar.

Verification Plan: We will **simulate bursts of edits** (e.g. edit 100 pages in a minute) and measure latency and queue buildup. We also plan an *end-to-end* test: edit one page, verify only that page’s triples changed. We should double-check that our NER linking indeed only re-links changed entities (sample by diffing). Finally, we will validate correctness by comparing full re-extraction vs incremental update on a test subset. We anticipate no bottleneck at 40K pages – but if edits peak, queuing may occur, so we might need a multi-worker setup or faster models.

Evidence strength: High – based on **DBpedia-Live** (one peer-reviewed paper ¹ and one official blog ⁷ describing incremental extraction).

Q9: Hybrid Semantic + Structural Clustering

Narrative: The optimal clustering combines content embeddings and graph structure. Empirical results suggest a *hybrid pipeline* (text-embedding + graph-embedding clustering) works best. For instance, Bastola & Choi (2025) report Top2Vec (semantic) + Node2Vec (graph) achieved silhouette ~0.927 vs 0.685 for text-only ². We will likely implement a two-step approach: first use a text embedding model (e.g. sentence-BERT or Top2Vec) to create dense vectors; cluster those (HDBSCAN/UMAP); then use a graph-embedding (Node2Vec or GraphSAGE) over the wiki link graph and cluster those; finally reconcile clusters (or intersect). Alternatively, we could use GraphSAGE to directly incorporate text features via neighbor aggregation.

Benchmarks/Params: Unfortunately, there is no benchmark on a *social-science wiki* corpus of our size. Bastola’s example was legal docs (one case study). We must do our own. For **Silhouette/DBI scores**, the cited hybrid pipeline gave Sil=0.927, DBI=0.110 ². As a sanity check, we’ll reproduce similar experiments on our wiki: try BERTopic (HuggingFace) vs Top2Vec for topic coherence (e.g. using `gensim.models.CoherenceModel`). For graph clustering, we can try **Leiden** (via `leidenalg` Python, active repo ⁹) on the page-link network. If communities don’t align, a hybrid embedding (concat text+graph features) might be needed (GraphSAGE or a graph neural net). Key hyperparams: e.g. BERTopic’s `min_cluster_size` ~10, HDBSCAN `min_cluster_size`; Top2Vec’s speed ~0.75 documents/sec on CPU; Node2Vec walks length ~10, context 10, dim=128; Leiden’s resolution ~1.0. We’ll sweep parameters and pick what maximizes silhouette or topic coherence (e.g. UMass score).

Named Libraries:

- **BERTopic** (Maarten Grootendorst) and **Top2Vec** for semantic clustering – both are maintained (recent commits late 2024–2025).

- **node2vec** (GitHub aditya-grover/node2vec) exists but is *stale* (last commit 2017) ¹⁰ ; we may instead use **PyTorch Geometric** or **StellarGraph** (GraphSAGE implementation, though that also lags) for graph embeddings.
- **GraphSAGE** official code is from 2018 (outdated) ¹¹ ; we could use StellarGraph (active) or DGL.
- **Leidenalg** (vtraag/leidenalg) is actively maintained (commits in 2026) ⁹ and can cluster with igraph.
- For evaluation: `sklearn.metrics.silhouette_score`, `sklearn.metrics.davies_bouldin_score`, and **Gensim CoherenceModel** for topic coherence.
- Implementation: use scikit-learn and huggingface `transformers` (BERT or Qwen). Use **NetworkX** or **igraph** for graph handling if needed.

Verification Plan: We must **evaluate clustering quality quantitatively**. We will compute Silhouette and DBI on held-out data and measure coherence of top words (via Gensim UMass or CV scores). We'll also manually inspect clusters to ensure interpretability. Crucially, because our graph is *heterophilic* (pages link often across topics), we must verify clusters truly capture semantic groupings, not just link communities. We'll test multiple parameter sets (e.g. varying number of clusters, embeddings) on a sample of our data. If pure graph clustering fails, the fallback is ensemble (e.g. nodes clustered together if either method agrees). At 40K scale, performance: Top2Vec claims ~0.7 docs/sec on 16GB CPU; Node2Vec (on ~40K nodes, 42K edges) should also run in minutes. We will measure runtime on a test machine. Finally, we'll ensure visual sanity by projecting clusters (UMAP) to see if similar pages group.

Evidence strength: Moderate – one strong case study (Bastola & Choi 2025, pre-print, peer-reviewed at AAAI or similar) ² and tool docs (deck.gl performance but unrelated). There is no third-party benchmark on our type of data, so we rely on that case study and partial anecdotal comparisons.

Q10: Wikidata Federation & Entity Alignment

Narrative: We align P2P-Wiki entities to Wikidata QIDs. The process is: extract list of entities (persons, organizations, concepts), then match to Wikidata via reconciliation tools. Tools include **OpenRefine** with the Wikidata reconciliation API, and **Mix'n'Match** for linking external lists. For example, we'd export all occurrences of "Elinor Ostrom" etc. and use OpenRefine's Wikidata reconciler (<https://op.refine.org>) to find QIDs. Mix'n'Match can be used if we prepare a CSV with these items. For bulk upload, **Wikidata Integrator (Python)** and **QuickStatements** can add or merge data.

Quality metrics: We have not found published precision/recall numbers for large-scale alignment. In practice, reconciliation is high-precision for well-known names (often $\geq 90\%$), but ambiguous cases require manual disambiguation. One case study (Nesić et al. 2022) used OpenRefine & QuickStatements to align 720 works to Wikidata, yielding 20K items ¹². They report careful manual checks ("necessity of manual checks") ¹². For our scale (maybe ~5K unique P2P entities), expect tens of hours of curator work for ambiguous matches.

Case studies: The **ELTeC** project (Nesić 2022) is a peer-reviewed example where 720 literary works were reconciled to Wikidata, creating ~20K items ¹². This shows the pipeline (OpenRefine→QuickStatements) can scale to several thousand. Other known efforts: "WikiCite" and museum collections have done 10K–100K linking, often via Mix'n'Match but many are unpublished. No well-documented 5K–20K study aside from ELTeC's mention.

Governance: Wikidata requires that large updates be community-reviewed. For thousands of entities, we should either request a **bot flag** (Wikidata:Requests for permissions/Bot) ¹³, or have trusted editors with access run QuickStatements manually. In practice, many bulk uploads use QuickStatements without bot (slow) or use a bot after RFC. The governance model: start an edit proposal on Wikidata-WikiProject (like notability) or Bot requests. The community expects sourcing for new statements. If we push new “instance of” or “same-as” claims, it should be reviewed or flagged by patrollers.

Named Tools:

- **OpenRefine** (reconciliation to WD) – widely used by librarians. (no specific commit check needed)
- **Mix’n’Match** – a WD tool for batch matching (we can prepare JSON catalogs).
- **WikidataIntegrator** (PyPI “wikidataintegrator”) for scripted QID writing (active project).
- **QuickStatements** (web/service by Wikidata, well-used, some repos exist).
- These are all real GitHub/PyPI: QuickStatements (M. Manske) - see github.com/magnusmanske/quick_statements. We should flag if outdated: QuickStatements is actively maintained. WDI (WikidataIntegrator) is active.

Verification Plan: To gauge accuracy, we will spot-check a random sample of 100 matched items, measuring percent correct. We will also compute alignment coverage (how many P2P names found in WD). For scale, we will stage an import of say 500 items, see if WD’s patrollers raise issues, and check load time (WD APIs are fast for single edits, bulk via Quickstatements should handle thousands). We can automate with WDI and run on a dev version first. If we find many conflicts (e.g. duplicates), we will refine match queries. Finally, we’ll liaise with the Wikidata community for review of our approach.

Evidence strength: Low/Moderate – based mainly on one case study ¹² (“one case study”) and tool documentation. No performance or quality benchmarks available; our plan will be empirically tested.

Q11: JSON-LD / SPARQL / Datasets Dump Pipeline

Narrative: We will export our wiki as a knowledge graph using best-practice vocabularies: e.g., **schema.org** (Article, Person, Organization, CreativeWork, etc.) plus **PROV-O** for edit provenance, **FOAF** for people, **SKOS** for categories, and a small custom P2P ontology (e.g. `:hasCategory`). The pipeline can reuse RDFizers: akin to Wikidata or DBpedia’s approach, but with our schema. A JSON-LD dump (one file or per-page file) can be published. We’ll provide a SPARQL endpoint (e.g. via Apache Jena Fuseki or GraphDB). We’ll deposit a Zenodo DOI and pin files on IPFS for immutability, and create a HuggingFace Datasets entry (markdown card) describing the RDF dataset and access methods.

Who’s done this at ~40K scale? We found **no published example** of a small wiki RDF dump aside from large projects. Relatedly, the *Wikipedia Observatory* and *Wikidata* produce huge dumps (Mgmacleod’s Wikidata5M-KG with 21M triples ¹⁴, Jotschi’s 17M Wikipedia KG ¹⁵). But a 40K-page wiki KG is an unusual case – none seem documented. We may note the **Wikidata JSON dumps** (100GB scale, but not curated JSON-LD) and schema crosswalks. The closest analogy is “DBpedia for a small wiki,” but DBpedia focuses on infobox extraction.

Costs: At 40K pages, the total triples might be modest (e.g. if each page yields ~100 triples, total ~4M triples). Converting to JSON-LD or TTL is cheap on modern servers (minutes). Serving SPARQL requires

memory; with Fuseki or Blazegraph on a few GB machine should be fine. We plan weekly or monthly updates, triggered by recent-changes. Hosting on Zenodo is free up to 50GB, so small costs. HuggingFace Datasets hosting is free. IPFS pin via Filecoin has minor fees (tens of \$ per year).

Consumer adoption: Hard to predict. Very few users likely: small wikis rarely consume others' dumps. As a reference, WikiData5M-KG (large) has ~40 HF downloads/month ¹⁶. Our release may attract some curiosity (maybe researchers), but it won't see heavy use unless specifically needed. We'll monitor Zenodo/HF metrics post-release.

Evidence strength: Low – no direct case studies found. We rely on analogies (Wikidata/HF data ¹⁵ ¹⁴) and general experience. This is more a built-as-needed task.

Q12: Visualization Stack at WebGL Scale

Narrative: For interactive graph viz of 40–100K nodes, we lean on GPU-based libraries. Our candidates:

- **deck.gl** (Uber): Known to handle ~1M scatter points at 60 FPS ³. At 40K nodes, deck.gl ScatterplotLayer or GraphLayer (if using edges) should be trivial. We can attach 2D positions (e.g. t-SNE) and render points.
- **regl-scatterplot**: explicitly optimized for millions of points ⁴. A WebGL scatterplot can easily do our node count.
- **Cosmograph**: reportedly can handle up to ~1 million nodes/edges ¹⁷ (though it's a special React app). It's a proof-of-concept; not a drop-in lib but shows feasibility.
- **sigma.js v3 + graphology**: Sigma's older benchmarks show GPU mode handles ~100K edges, but gets slow beyond ~50K edges ¹⁸. For our ~42K edges, it might run at ~30–60 FPS depending on desktop GPU. We'd prefer WebGL line/point rendering. Sigma's community iteration works, but it's lower level.
- **three.js custom**: One can code a custom WebGL renderer with Points/Lines. Three.js can render 100K points (Particle systems) easily (we have used ~50K in demos).
- **Kepler.gl**: Uses deck.gl under the hood; designed for geodata. Not ideal for generic graph.
- **Observable/retina.js**: retina.js (by Jacomy/MITRE) can do 100K nodes, but it's AngularJS-era and likely outdated.

Non-force-directed views:

- **Voronoi drill-down**: using D3's Delaunay/Voronoi (via d3-delaunay) to cluster points into regions. It can handle ~10–20K points for interactive use; beyond that performance may drop. But cluster-level drill-down might aggregate points.
- **Sankey diagrams** (d3-sankey): For claim flows/category flows, typical use is hundreds of nodes (flows) not tens of thousands. We'd pre-aggregate flows by category pair to ~100 links, else performance becomes poor (e.g., Chart shows ~200 nodes/edges fine).
- **Chord diagrams** (d3-chord): Best for small matrices (<50x50). Our category co-occurrence graph (maybe 1000 categories x 1000) would need heavy filtering (e.g. show top 50).
- **Dendrogram (Leiden hierarchy)**: Use d3-hierarchy or Cluster functions. 40K leaves is too many to view at once; we'd likely only visualize top-level branches, letting user expand.

- **Timeline of emergence:** Could use Vega-Lite or Google Charts (temporal scatter/line). E.g. a time-series plot (Ngram style) is trivial for 18 years (18 points) – easy.
- **Bioregional choropleths:** If mapping categories to regions, use deck.gl or Leaflet with GeoJSON overlay.

Performance: Citing deck.gl docs ³ : 1M GPU points at 60 FPS on a 2015 MacBook, dropping to ~10–20 FPS at 10M. Our 40K is two orders of magnitude below 1M, so we expect 60 FPS on mid-range hardware. Regl-scatterplot explicitly claims ~20M points ⁴ . Cosmograph's info ¹⁷ suggests 133K nodes+321K edges at ~30–60 FPS. Sigma.js can handle our scale but will require careful tuning (avoid heavy label rendering).

Named Libraries:

- **deck.gl** (<https://github.com/visgl/deck.gl>) – active repo, GL-based layers.
- **regl-scatterplot** (<https://github.com/regl-project/regl-scatterplot>) – active, WebGL scatter.
- **Graphology + sigma.js v3** – graphology (JS library), sigma.js v3 (<https://github.com/jacomyal/sigma.js> – active commits).
- **cosmograph** – not a published lib, skip linking.
- **D3** for Sankey (d3-sankey), Chord (d3-chord), Voronoi (d3-delaunay).
- **three.js** for any custom 3D/2D.
- We should note “deck.gl’s official perf doc” ³ and “sigma vs ogma blog” ¹⁸ as evidence.

Verification Plan: We will prototype key views and measure FPS/memory. For example, load a real subgraph of ~50K nodes/50K edges into deck.gl and sigma, measure using Chrome’s dev tools on a mid-tier machine. For Voronoi/Sankey, test d3’s performance with our data sizes; if too slow, reduce via sampling. We’ll ensure interactive responses stay >30 FPS for desktop (lower for mobiles is acceptable). We’ll also profile memory (Chrome about:memory or Firefox profiler) to ensure no leaks. If graphs exceed capabilities, we will implement level-of-detail or clustering (e.g. aggregate nodes into meta-nodes for overview).

Evidence strength: Moderate – based on vendor docs and blog posts ³ ⁴ ¹⁷ ¹⁸ . These are credible (library documentation, developer blogs). We will still verify on our exact data.

Q13: UMAP/PaCMAP/TriMap for 100K Embeddings

Narrative: For projecting ~96K embedding chunks to 2D/3D, we need scalable dimensionality reduction. **UMAP** is a strong default (used by many viz tools). We’ll use an efficient implementation (e.g. [UMAP](#) or NVIDIA’s cuML GPU-UMAP). The HuggingFace Embedding Atlas example uses UMAP with `n_neighbors=30, min_dist=0.1` ¹⁹ , which is a reasonable start. **PaCMAP** and **TriMap** can be tried: TriMap is known for preserving global clusters but can be slower, PaCMAP for speed. All three are stochastic, so repeat runs will vary. To ensure stability, we will fix random seeds and possibly run multiple times to check variability. In practice, minor visual jitter is acceptable.

Rendering: Once 2D coords are computed offline, we can serve them via deck.gl scatter or regl scatter (as in Q12). For 3D, we can use `three.js` or deck.gl’s 3D capabilities (on WebGL2). TF Embedding Projector (TensorBoard) historically only loads first 100k points ²⁰ , so we’re at that limit; we need a custom viewer (deck.gl custom OR embedding-atlas like HF’s, which can handle millions ²¹). Embedding Atlas (HuggingFace) claims smooth interaction on millions ²¹ ; it presumably uses WebGL behind the scenes.

Examples: - **Nomic Atlas** (docs suggest using UMAP) ²² ¹⁹. They mention configuring UMAP as above. - **TensorFlow Projector** caps at 100k ²⁰, so not ideal. - **Lilac** (by Nomic) and **OpenAI Atlas** (Apple/HuggingFace) use similar tech but no public repos. HuggingFace's Embedding Atlas (prod) specifically advertises "smooth, interactive millions" ²¹, implying GPU/WebGL. - **Gensim projector** likely uses TFJS and also hits 100k cap.

Parameter Regimes: Based on the above, UMAP with $n_neighbors \approx 15-50$ and $min_dist=0.0-0.1$ often yields tight clusters. The HF Atlas example used 30/0.1 ¹⁹. We will experiment (e.g. $n_neighbors$ from 15 to 50, min_dist 0.0 to 0.2). PaCMAP parameters: for 100k, maybe $K=30$ neighbors and 10 nearest for mid (common defaults). TriMap: often use default 20k triplets sample. We'll measure run-time: UMAP (RAPIDS/GPU) can do 100k in seconds ²³; CPU UMAP might take minutes (but acceptable offline). We'll also test stability by running 3 times and comparing cluster alignments.

Cost of recomputation: On corpus growth, adding new chunks may require full recompute (especially UMAP) for consistency. If we have millions of points this is expensive. But at 100k scale, a weekly rebuild is feasible (~seconds on GPU). If needed, we could do incremental updating (e.g. UMAP's transform of new points) but likely we'll just regenerate embedding on each update batch (say daily).

Rendering: We will likely use **deck.gl** with `ScatterplotLayer` for 2D (requires flattening to screen coords), and **scatterplot-renderer** or **three.js** for 3D (camera controls). The embedding results must be sent to client (via JSON or binary). We should ensure that the coordinate data is only transmitted once (or cached), and use WebGL instanced drawing for speed.

Evidence strength: Moderate – based on tool documentation and example configs ²⁰ ²³ ²¹ ¹⁹. We will verify with our data and measure times.

Q14: Concept-Drift Visualization

Narrative: We want to trace when concepts like "commons" and "bioregion" intersect in the literature. Techniques: train word embeddings per time-slice (e.g. yearly) to see semantic drift ²⁴, or run **Dynamic Topic Models** (Blei 2006) to capture evolving topics. For simple frequency-based trends, a line chart of co-occurrence counts per year (like Google Ngram) is useful. For richer patterns, we can use timeline of topic word probabilities (via gensim's `LdaSeqModel` ²⁵) or diachronic embeddings (HistWords method ⁵).

Tools: - **HistWords** (Hamilton et al. 2016) is a toolkit for diachronic word2vec; it also came with a simple interactive viz where you plot word meaning changes ⁵. For UI, the HistWords web demo shows timeline plots for words (like "awful vs terrible") ⁵ – we could mimic that to plot "commons" and "bioregion" trajectories. - **Dynamic Topic Model (DTM):** Gensim's `LdaSeqModel` implements DTM ²⁵. We can train it on annual bins of our corpus to see when a topic including both terms emerges. - **DETM (Dynamic Embedded Topic Model):** a deep learning approach, though complex to implement. - **PubMed Trends:** (Lewitus et al. 2018) shows co-occurring phrases trends via plots, not publicly available tools. - **Google Ngram architecture:** uses simple multi-line plots over time. They likely use bigram counts per year with a back-end SQL; for our case, simply count pages per year with both terms.

UI Primitives: No out-of-the-box library specifically for conceptual drift in wikis. We propose:

- **Line Charts:** Plot frequency of keywords or co-occurrences vs year (like NgramViewer).
- **Color/Wordcloud Timelines:** For a given term, display related words per time-bin (like "word shift" diagram).
- **Embedding Evolution:** 2D scatter where each point is a year-location of a word vector (similar to HistWords screenshot ⁵).
- **Alluvial/Sankey:** Show how topics merge/split: e.g. topic containing "commons" flows and at some year merges with one containing "bioregion".
- **Interactive Sliders:** Time slider controlling displayed clusters or wordsets.
- **Dashboard:** Perhaps an all-in-one showing a line chart, an embedding scatter, and key phrases.

We found the HistWords example particularly illustrative: it lets users select multiple words and shows their embeddings over time in a timeline view ⁵. Similarly, Google's Ngram Viewer shows frequency lines and allows up to 5 terms comparison. We didn't find any wiki-oriented tool, so custom is needed.

Evidence strength: Low – some methods (HistWords, DTM) are published (peer-reviewed ²⁴ ²⁵), but no known UI/library specifically does this. UI suggestions are drawn from analogous tools (Ngram viewer, topic modeling apps). We'll prototype charts (e.g. D3 line charts) and see if they convey the drift clearly.

Q15: Sheaf-Cohomology Visualization

Narrative: Sheaf cohomology (H^1) marks "tensions" where local claims conflict globally. To visualize, we might highlight nodes or edges involved in non-zero cocycles. However, no existing tool does exactly this. We looked at TDA libraries: **pyflagser** (GPU flag complex) and **Gudhi** (persistent cohomology) compute cohomology, but their outputs are mostly algebraic (diagrams). They lack a user-facing "heatmap" UI. We found no prior example of a dashboard for sheaf disagreement. The closest analog is general **persistent homology visualization**: barcodes and heatmaps that show where cycles appear. For example, *gtda* (Giotto-tda) added `FlagserPersistence` (via *pyflagser*) ²⁶, but it only outputs persistence diagrams, not spatial graphs.

We did find a conceptual description: "Cohomology acts like an automated cross-referencing system... it flags global inconsistency (a non-zero 1st Cohomology)" ²⁷. This supports the idea that H^1 highlights contradictions. In absence of ready UI, we may build a custom graph overlay: e.g., color edges by magnitude of cocycle values, or highlight "disagreement edges" where incident pages strongly contradict. Another idea: an interactive table listing edges with highest "tension scores." However, no library or paper shows a prototype of such visualization. Even in Federated IRT or social graph mining, disagreement graphs are not standard.

Closest analogs: General topology tools like **HERA** (for persistent distance) or **Gudhi** can draw betti-1 barcodes, but not tied to nodes. No published "disagreement dashboard" found. Possibly related work: some distributed consensus literature visualizes difference networks, but no UI references. The "Moving Beyond Graphs" blog ²⁸ explains H^1 conceptually, but no GUI. The **persistence barcode** idea is the nearest: one could color-code edges by "importance in 1-cocycles" similar to how Betti-1 bars measure cycle lifespan. But that's speculative.

Implementation: We may need to prototype a UI: for instance, overlay on the wiki graph a heatmap where edge color/intensity $\propto$ disagreement measure (if defined). Or a slider on a topological “time” axis: at threshold t , edges with $\text{cocycle} \geq t$ are shown. Alternatively, produce a “graph of contradictions”: connect pages if they disagree (like citation contradiction graphs in SciScore). None are known off-shelf.

Evidence strength: Very low – mainly conceptual. We cite a Medium blog ²⁷ for idea validation. No known software exists to adapt. We will likely use tools like **plotly** or D3 to draw our custom visualizations, and base them on computation from pyflagser (H^1 cocycles) or our Claims Engine. In verification, we’d test on small subgraphs with known contradictions to ensure highlights appear correctly.

Q16: Citation Verification Layer

Narrative: We need an AI-assisted citation checker. The most relevant system is **SIDE (2023)** ⁶ (Meta research), which flags likely-unsupported Wikipedia citations and suggests replacements. It’s an LLM+retrieval model trained on wiki sentences. Petroni et al. report that for the top 10% of “unverifiable” citations, human evaluators *preferred* the system’s suggestion ~70% of the time ²⁹. This is promising but the code is not yet production-ready (it was a research demo).

Alternatives:

- **Citely.ai** (AI citation context summarizer) – commercial, unclear reliability, no public docs on performance.
- **SciScore** – a journal manuscript tool, not directly for links; likely irrelevant here.
- **Scite.ai** – provides an API that labels academic citations as “supporting/contradicting/unrelated.” Could query DOIs or PMIDs. It’s a stable product with an API. It won’t catch everything (e.g. news sources).
- **Crossref Event Data** – open API of citation events (paper-to-paper links, references). Not a checker, but could fetch how often a cited DOI is cited/supported.
- **Retraction Watch API** – free service listing retracted publications. Useful to flag “bad” citations if a cited DOI is retracted. The API (<https://api.retractiondatabase.org>) is public.
- **InternetArchiveBot** – Wikipedia bot that archives dead URLs. This increases link persistence but doesn’t check content validity. It’s not “verification” of truth, just link rot prevention.

For a 40K-page wiki:

- Checking thousands of references is feasible. Using *Scite.ai* with its free tier (250 requests/day) is slow; their paid tier can speed it up but costs unknown. Retracted DOIs: a few hundred k entries are free. Crossref ED: free and unlimited queries for reference events.
- **False positives:** Models like SIDE or Scite could mis-label a legitimate source as “unsupported” or vice versa. That’s why human vetting is needed. For instance, SIDE’s 70% success suggests 30% might disagree.
- **Governance:** Any flagged “bad” citations ultimately require human judgment. We could tag suspicious ones (e.g. with a Template or a flag for review). The policy is that Wikipedia community decides verifiability – a tool can only assist by flagging. For batch use, a bot would need community approval. Likely workflow: generate a report or use bots to tag “Citation Needed” on flagged claims, then let editors resolve.

Named Tools: SIDE (the implemented code has no public repo yet as of 2024, but the paper is Nature Machine Intell) ⁶ ²⁹ . Scite.ai (Commercial; API docs available). Retraction Watch API (open, free). No “official” MediaWiki extension found for side-by-side citation checking. The Wikipedia InternetArchiveBot and Citation Bot do archival but not mis-info. There’s an old extension **Citoid** that formats references, but not a validator.

Verification Plan: We’ll prototype on a sample of pages: run Scite on DOIs and see how many get “contradicting” results (perhaps flag if contradictions > 50%). Use Crossref ED to list citing papers (an unexpected citation context may hint a problem). Use SIDE’s demo if available (it’s not). Each flagged case we’ll manually inspect. We’ll also track how many alerts each tool raises: e.g. retracted DOIs (Crossref+RW) vs time. We must define thresholds (e.g. if Scite says “majority citing contradicts this claim, flag”). We’ll then have editors review, and measure false positive rate by sampling flagged citations versus true issues.

Evidence strength: Moderate – based on **one peer-reviewed study (SIDE)** ⁶ ²⁹ and known services (Scite, Retraction Watch). SIDE’s results (70% preferred) are strong evidence for benefit. Other tools’ reliability is anecdotal. We treat all automated flags as “tools, not truth”: community judgment is ultimate arbiter.

Q17: MediaWiki AI Extensions in Production

Narrative: We inventory AI-related extensions and check their maturity.

- **NeoWiki:** (ProfessionalWiki) Experimental AI editor. Actively developed (multiple commits in April 2026 ³⁰ by several devs). It provides structured content editing with AI. It’s not officially released on mediawiki.org, and seems used only on ProWiki-hosted wikis. **Health:** High recent activity, multi-person team. **Deployment:** Only on ProWiki (unknown total pages). No evidence of use on 10K+ page wikis.
- **Wanda:** (Techwizzie) Open-source chatbot. Latest version 1.0.0 released Oct 2025 ³¹ , actively maintained by one author. **Health:** Single maintainer but stable release; open-source (GPL). Likely lightweight adoption (requires Elasticsearch). Not aware of large deployments (maybe R&S).
- **AI Assistant (ProWiki):** Commercial chatbot (ProWiki), stable use on ProWiki’s hosted sites. We have a description ³² . **Health:** Actively marketed (ProWiki commercial product). No public repo; likely not community-developed.
- **Extension:KnowledgeGraph:** Actually an SMW graph viz tool ³³ , not an AI content tool. (Probably irrelevant to this question.)
- **Extension:VIKI:** Unmaintained since 2017 ³⁴ , explicitly marked “not actively maintained” and recommended replacement. Not used on large wikis.
- **Extension:Cargo:** Mature data extension ³⁵ , widely used (see CargoTables at Wikimedia AU). Actively maintained (MediaWiki core team), clearly scaled to large data. But not an “AI” extension per se.
- **Semantic MediaWiki (SMW):** Very mature, large deployments. But LLM integration is mostly experimental (AskWikidata was abandoned). SMW is stable, no LLM-specific components out-of-the-box.
- **WandaScore/WandaScribe:** Spin-offs of Wanda – Scoring and Scribing assistants (listed on MediaWiki “Chatbots” page). Unclear if production-deployed anywhere. Likely less mature.
- **AIEditingAssistant:** Provided by Hallo Welt! (BlueSpice). Last LTS version 2.0.4, supporting MediaWiki 1.43 ³⁶ . Stable release, used in BlueSpice (which serves corporate wikis). **Health:**

Maintained by a company, multi-year support. Possibly used on wikis of unknown size (BlueSpice customers).

- **AskAI**: (from professionalwiki.com blog) – in development ³⁷, not stable.
- **ChatGPT**: No official MW extension found. However, AIEeditingAssistant uses ChatGPT API to suggest edits ³⁸ (see “highlight text and have ChatGPT make suggestions”). The extension name changed to “AIEeditingAssistant” but functions as a ChatGPT integration.

Battle-tested at 10K+?:

- Cargo: yes (many large instances).
- SMW: yes (established in Wikimedia).
- AIEeditingAssistant: deployed in enterprise wikis (size unknown, but likely thousands of pages).
- Wanda/AI chatbots: we found no published deployment stats. They seem aimed at smaller projects (KolZchut = a law portal, Prowiki blogs).
- NeoWiki: no published stats; likely only a few wikis.

Community health:

- NeoWiki: active dev (Apr 2026) ³⁰, bus factor ~2.
- Wanda: one maintainer, last release Oct 2025 ³¹, bus factor 1.
- AIEeditingAssistant: stable LTS, maintained by HalloWelt (commercial; bus factor small, but brand support).
- Others (VIKI) are stale (no commits since 2017 ³⁴) and thus not reliable.

Provenance Labels: None of these extensions inherently mark AI edits with provenance. To comply with attribution, we must add labels ourselves. For example, we can configure an `#AIAssist` template or an “AI-generated” flag on edits. The AIEeditingAssistant could be extended to append “*Suggest: [prompt]*” attribution. Chatbot answers (Wanda, AI Assistant) already include source citations by design ³² ³⁹. We will ensure any automated insert includes a footnote like “Generated by AI on [date]” as recommended.

Evidence strength: Mixed. We rely on official extension pages and professional blog posts ³² ³⁶ for maturity. Some evidence is “one extension page” or “official docs” which is medium-strength. No peer-reviewed data here, just code activity.

¹ ⁸ jens-lehmann.org

http://jens-lehmann.org/files/2012/program_el_dbpedia_live.pdf

² Hybrid Topic-Semantic Labeling and Graph Embeddings for Unsupervised Legal Document Clustering

<https://arxiv.org/html/2509.00990v1>

³ Performance Optimization | deck.gl

<https://deck.gl/docs/developer-guide/performance>

⁴ GitHub - flekschas/regl-scatterplot: Scalable WebGL-based scatter plot library build with Regl · GitHub

<https://github.com/flekschas/regl-scatterplot>

⁵ ²⁴ visualizing diachronic word embeddings

<https://oky.moe/histwords-explorer/>

- 6 29 Improving Wikipedia verifiability with AI | Nature Machine Intelligence
https://www.nature.com/articles/s42256-023-00726-1?error=cookies_not_supported&code=b298b399-82bf-40f5-b8e3-f1838a7368a0
- 7 DBpedia Live Restart - Getting Things Done - DBpedia Association
<https://www.dbpedia.org/blog/dbpedia-live-restart-getting-things-done/>
- 9 Commits · vtraag/leidenalg · GitHub
<https://github.com/vtraag/leidenalg/commits/main/>
- 10 GitHub - aditya-grover/node2vec · GitHub
<https://github.com/aditya-grover/node2vec>
- 11 Commits · williamleif/GraphSAGE · GitHub
<https://github.com/williamleif/GraphSAGE/commits/master/>
- 12 Bridging the Gaps: Integrating Bibliographic Metadata Into Wikidata for Literary Corpora | Journal of Open Humanities Data
<https://openhumanitiesdata.metajnl.com/articles/10.5334/johd.483>
- 13 Wikidata:Bots - Wikidata
<https://www.wikidata.org/wiki/Wikidata:Bots>
- 14 16 Alphonse7/Wikidata5M-KG · Datasets at Hugging Face
<https://huggingface.co/datasets/Alphonse7/Wikidata5M-KG>
- 15 Jotschi/wikipedia_knowledge_graph_en · Datasets at Hugging Face
https://huggingface.co/datasets/Jotschi/wikipedia_knowledge_graph_en
- 17 How to Visualize a Graph with a Million Nodes | Nightingale
<https://nightingaledvs.com/how-to-visualize-a-graph-with-a-million-nodes/>
- 18 Ogma | Ogma vs sigma.js: which graph visualization library for your application?
<https://doc.linkurious.com/ogma/latest/compare/sigmajs.html>
- 19 21 Embedding Atlas · Hugging Face
<https://huggingface.co/docs/hub/datasets-embedding-atlas>
- 20 tensorflow - Tensorboard Embedding projector Maximum points showing? - Stack Overflow
<https://stackoverflow.com/questions/43488919/tensorboard-embedding-projector-maximum-points-showing>
- 22 How to Visualize Embeddings with t-SNE, UMAP, and Nomic Atlas | Nomic Platform Documentation
<https://docs.nomic.ai/atlas/embeddings-and-retrieval/guides/how-to-visualize-embeddings>
- 23 Even Faster and More Scalable UMAP on the GPU with RAPIDS cuML | NVIDIA Technical Blog
<https://developer.nvidia.com/blog/even-faster-and-more-scalable-umap-on-the-gpu-with-rapids-cuml/>
- 25 models.lidaseqmodel - Dynamic Topic Modeling in Python — gensim
<https://radimrehurek.com/gensim/models/lidaseqmodel.html>
- 26 Release Notes — giotto-tda 0.5.1 documentation - GitHub Pages
<https://giotto-ai.github.io/gtda-docs/latest/release.html>
- 27 28 Moving Beyond Graphs — The Hidden Mathematical Revolution Investors Should Understand Now | by Devansh | Medium
<https://machine-learning-made-simple.medium.com/moving-beyond-graphs-the-hidden-mathematical-revolution-investors-should-understand-now-8883e3ce4aa5>

30 Commits · ProfessionalWiki/NeoWiki · GitHub

<https://github.com/ProfessionalWiki/NeoWiki/commits>

31 Extension:Wanda - MediaWiki

<https://www.mediawiki.org/wiki/Extension:Wanda>

32 37 MediaWiki Chatbot Extensions Compared - Professional Wiki

<https://professional.wiki/en/articles/mediawiki-chatbot-extensions-compared>

33 Extension:KnowledgeGraph - MediaWiki

<https://www.mediawiki.org/wiki/Extension:KnowledgeGraph>

34 Extension:VIKI - MediaWiki

<https://www.mediawiki.org/wiki/Extension:VIKI>

35 Extension:Cargo - MediaWiki

<https://www.mediawiki.org/wiki/Extension:Cargo>

36 38 Extension:AIEditingAssistant - MediaWiki

<https://www.mediawiki.org/wiki/Extension:AIEditingAssistant>

39 Chatbots - MediaWiki

<https://www.mediawiki.org/wiki/Chatbots>